

APPLICATIONS OF LARGE LANGUAGE MODELS

AME WINTER PROJECT

Made by -
Abhinav Sinha
Dhruva Ranjan Datta
Harsh Shekhawat

TASK 1

The first task was to design and optimize a Convolutional Neural Network (CNN) for multi-class classification using a publicly available dataset. We had to experiment with architecture changes, hyperparameters, and data augmentation. Evaluation of the the model using metrics like accuracy, precision, recall, and F1-score was to be shown, while documenting all steps, challenges, and insights.

The main learning from this task involves understanding and applying the key principles of deep learning, specifically using Convolutional Neural Networks (CNNs).

It develops a comprehensive understanding of building, training, evaluating, and optimizing a CNN model for a real-world classification problem. The focus on iterative improvement, detailed evaluation, and clear documentation fosters not only technical expertise but also analytical and problem-solving skills.

Data Preparation and Analysis

Building CNN Architecture

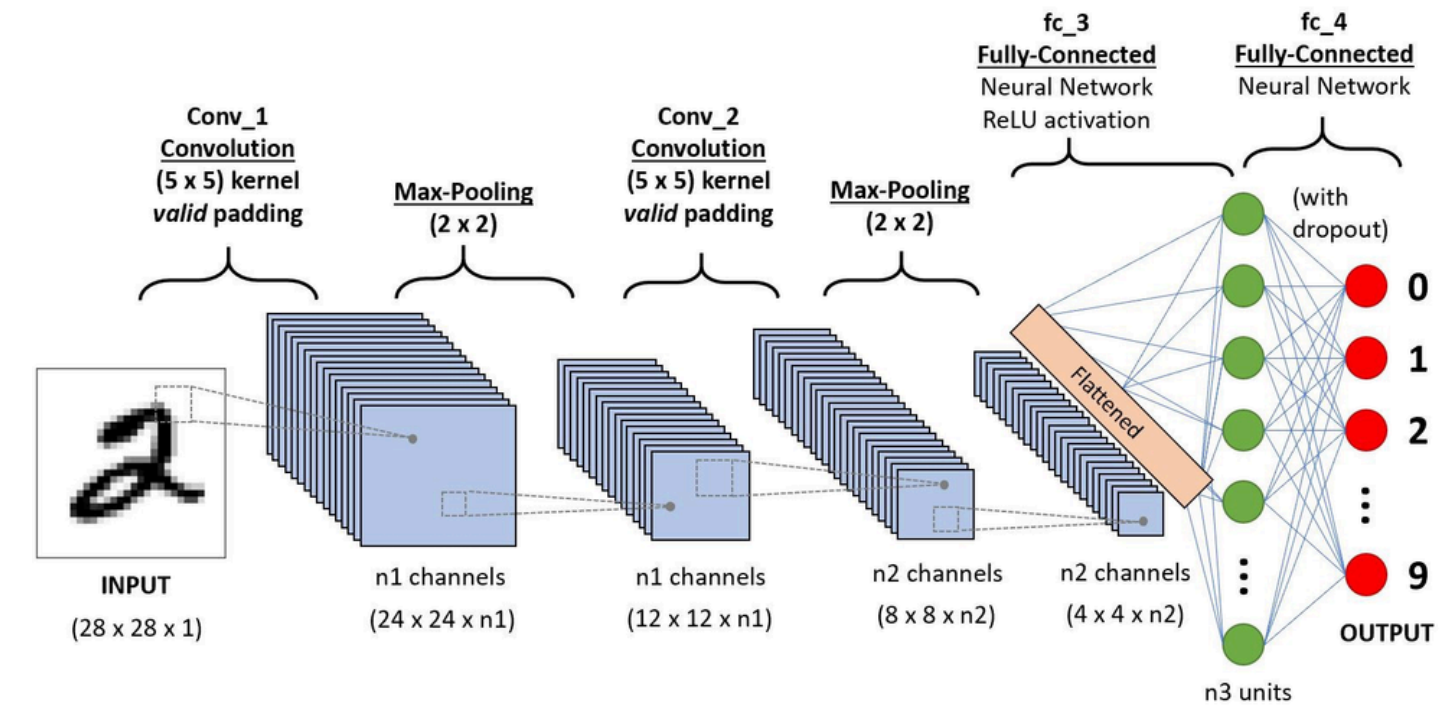
Performance Evaluation

Experimentation and Optimisation



Convolutional Neural Networks

Basic CNN Structure



```
model = tf.keras.models.Sequential()

model.add(tf.keras.layers.Conv2D(filters=32, kernel_size=(3,3), activation='relu', input_shape=(28, 28, 1)))
model.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2)))

model.add(tf.keras.layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu', input_shape=(28, 28, 1)))
model.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2)))

model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(28, activation='relu'))
model.add(tf.keras.layers.Dense(28, activation='relu'))
model.add(tf.keras.layers.Dense(10, activation='softmax'))
```

Code Snippet
from Task

TASK-2

In our second task we learnt how to apply LSTM models on time-series data and explore how changes in model architecture and hyperparameters affect the results. We started with the provided sample code, which used LSTMs to predict sequential data, and then went on to experiment with the model architecture by adding more LSTM layers and dense layers, adjusting the number of neurons, and incorporating Dropout layers to study their impact on the model's complexity and performance.

From this task we learnt about RNNs and LSTMs. Their architecture and working, how they are very useful in processing sequential data and how we can apply it on real world data.

```
[ ] # create and fit the LSTM network
model = Sequential()
model.add(LSTM(4, return_sequences=True, input_shape=(1, look_back)))
model.add(LSTM(8, return_sequences=True))
model.add(LSTM(8))
model.add(Dense(8, activation='relu'))
model.add(Dense(4, activation='relu'))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='rmsprop')
model.fit(trainX, trainY, epochs=75, batch_size=10, verbose=2)
```



RECURRENT NEURAL NETWORKS(RNNs)

While neural networks are great, they are not very useful when it comes to sequential data processing. It is because of the following reasons

- Neural networks do not have memory, they cannot retain information from previous inputs. So they are not quite helpful in tasks which require “context”.
- Neural networks have fixed input size, so they won't be able to handle variable length sequences.

This is where RNNs come into picture. RNNs are a type of neural network that can model sequential data. The special features of RNNs are :-

- **Internal memory**- Allows them to remember past inputs and use that context when processing new information.
- **Sequential Data processing**- Because of their memory, RNNs are exceptional at handling sequential data where the order of elements matters. This makes them ideal for speech recognition, machine translation, natural language processing (NLP) and text generation.



RNN ARCHITECTURE

INPUT LAYER

This layer receives the initial element of the sequence data.

OUTPUT LAYER

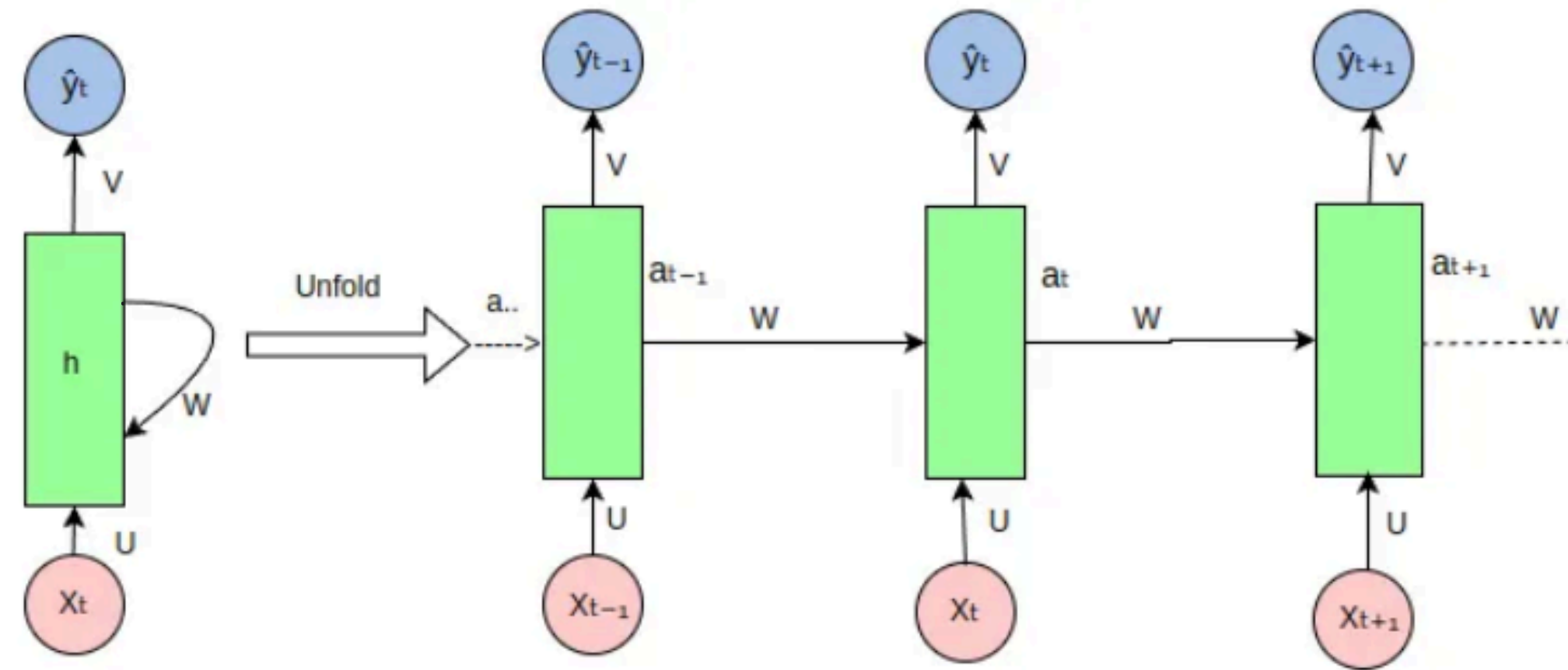
The output layer generates the network's prediction based on the processed information.

HIDDEN LAYER

The heart of the RNN, the hidden layer contains a set of interconnected neurons. Each neuron processes the current input along with the information from the previous hidden layer's state. This "state" captures the network's memory of past inputs, allowing it to understand the current element in context.

RECURRENT CONNECTION

A key distinction of RNNs is the recurrent connection within the hidden layer. This connection allows the network to pass the hidden state information (the network's memory) to the next time step



LSTMs

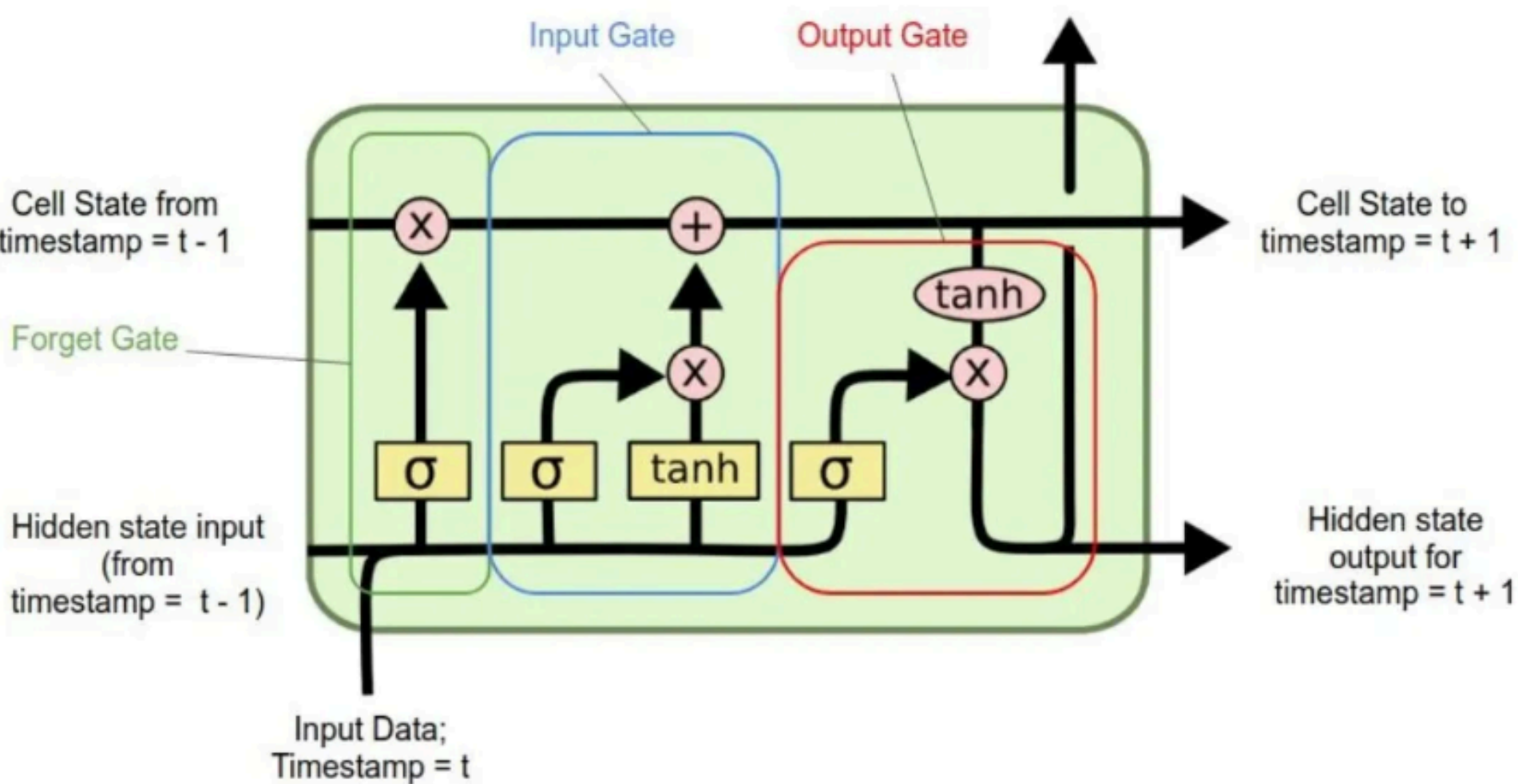
LSTMs come to the rescue to solve the vanishing gradient problem. It does so by ignoring (forgetting) useless data/information in the network. The LSTM will forget the data if there is no useful information from other inputs (prior sentence words). When new information comes, the network determines which information to be overlooked and which to be remembered.

LSTMs have 4 different components, namely

1. Cell state (Memory cell)
2. Forget gate
3. Input gate
4. Output gate

LSTMs, instead of just a simple network with a single activation function, we have multiple components, giving power to the network to forget and remember information.

LSTM ARCHITECTURE



LSTM components; Source: [Turing's blog](#)

Cell State is the long term memory. The forget gate decides what fraction of this long term memory should be remembered and what should be forgotten. There are no weights on the long term memory so as to remove the vanishing gradient problems.

Input gate gives new information to the LSTM and decides if that new information is going to be stored in the cell state. The new cell state $C(t)$ is obtained by adding the output from forget and input gates. The output of the LSTM unit depends on the new cell state.

Task 3

Encoder-Decoder Models Using LSTMs/RNNs

Tuning Neural Network Hyperparameters and Layers

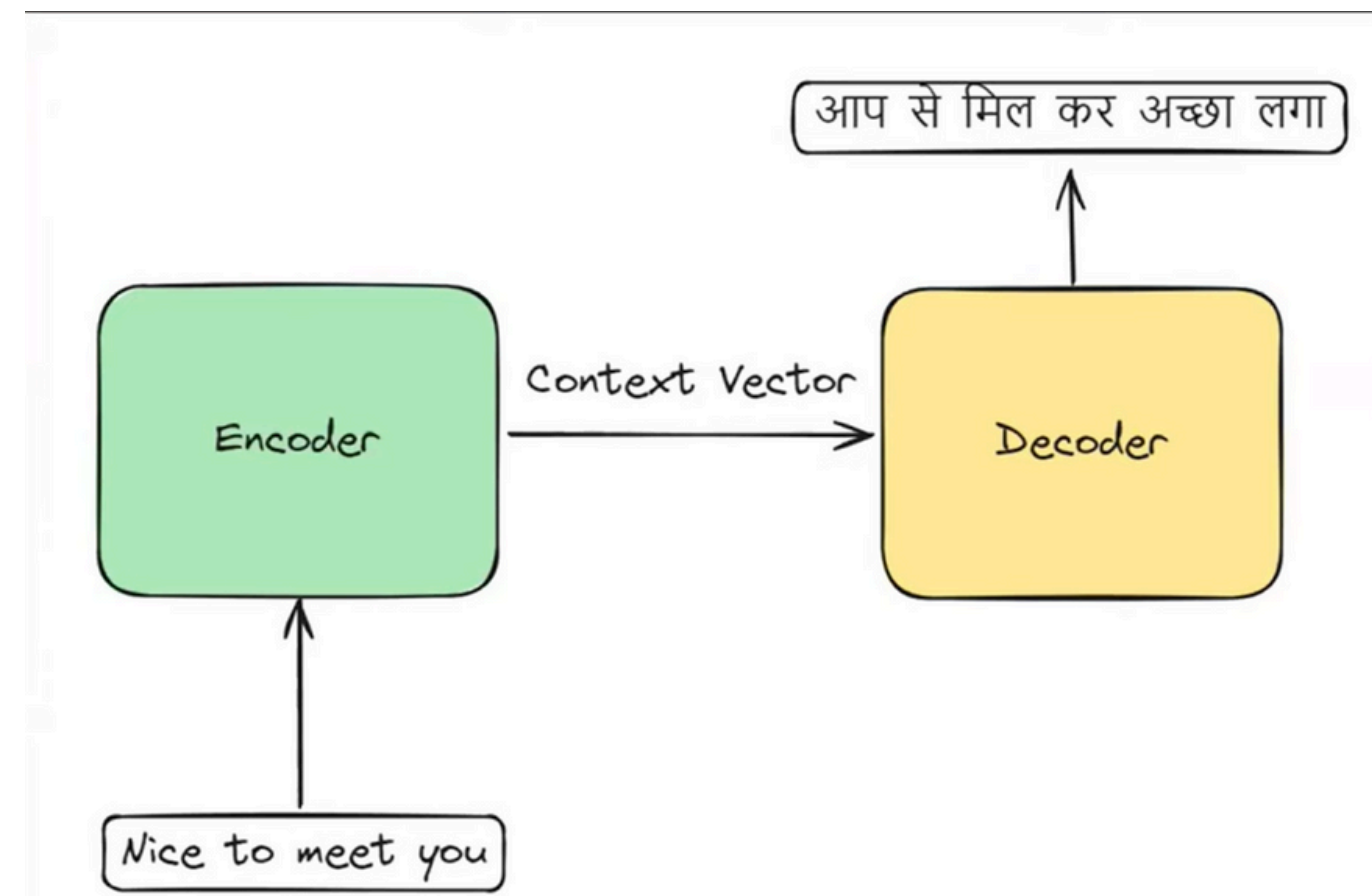


Encoder – Decoder Models

The Encoder–Decoder architecture is a framework for handling sequence-to-sequence tasks, such as translation and summarization. It maps variable-length input sequences to variable-length outputs using two main components.

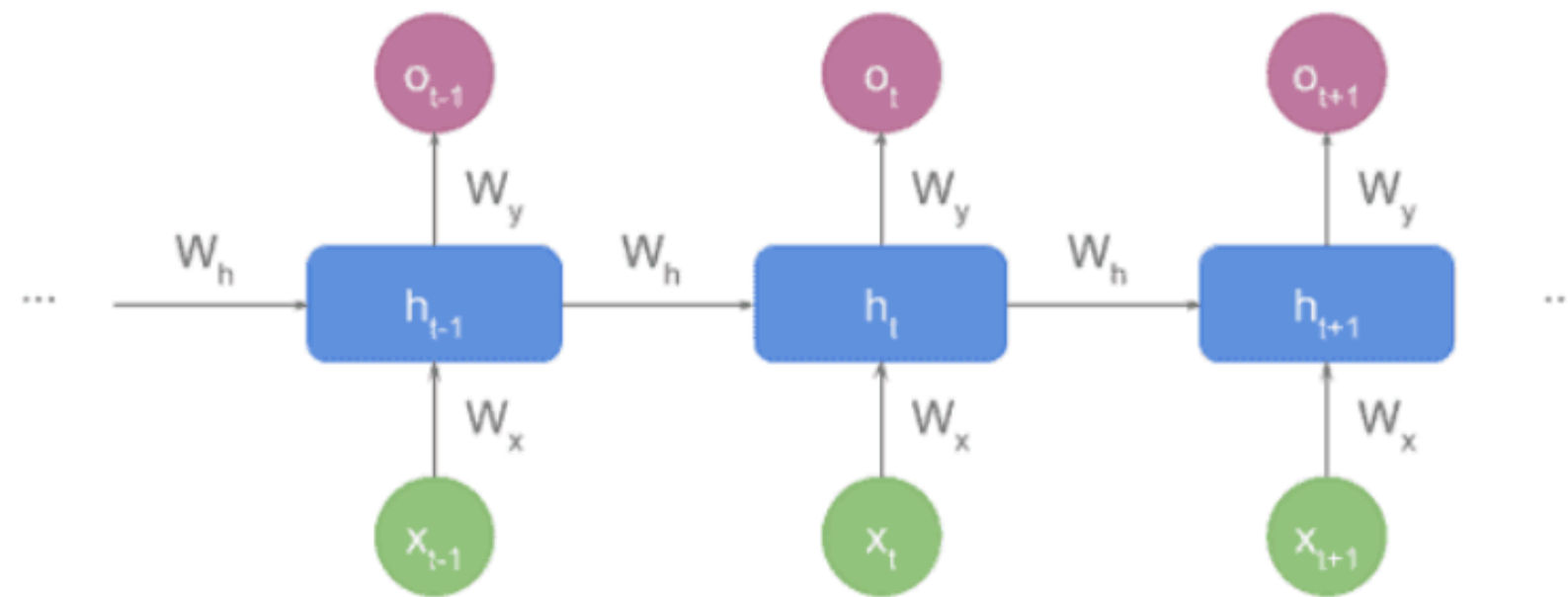
Encoder: Processes the input sequence and generates a fixed-size context vector summarizing its information.

Decoder: Takes the context vector and generates the output sequence one token at a time.



How It Works

1. The encoder reads the input sequence, updating its hidden state step-by-step to produce the context vector.
2. The decoder uses the context vector to initialize its state and iteratively predicts the output tokens, with each prediction informed by the previous token.

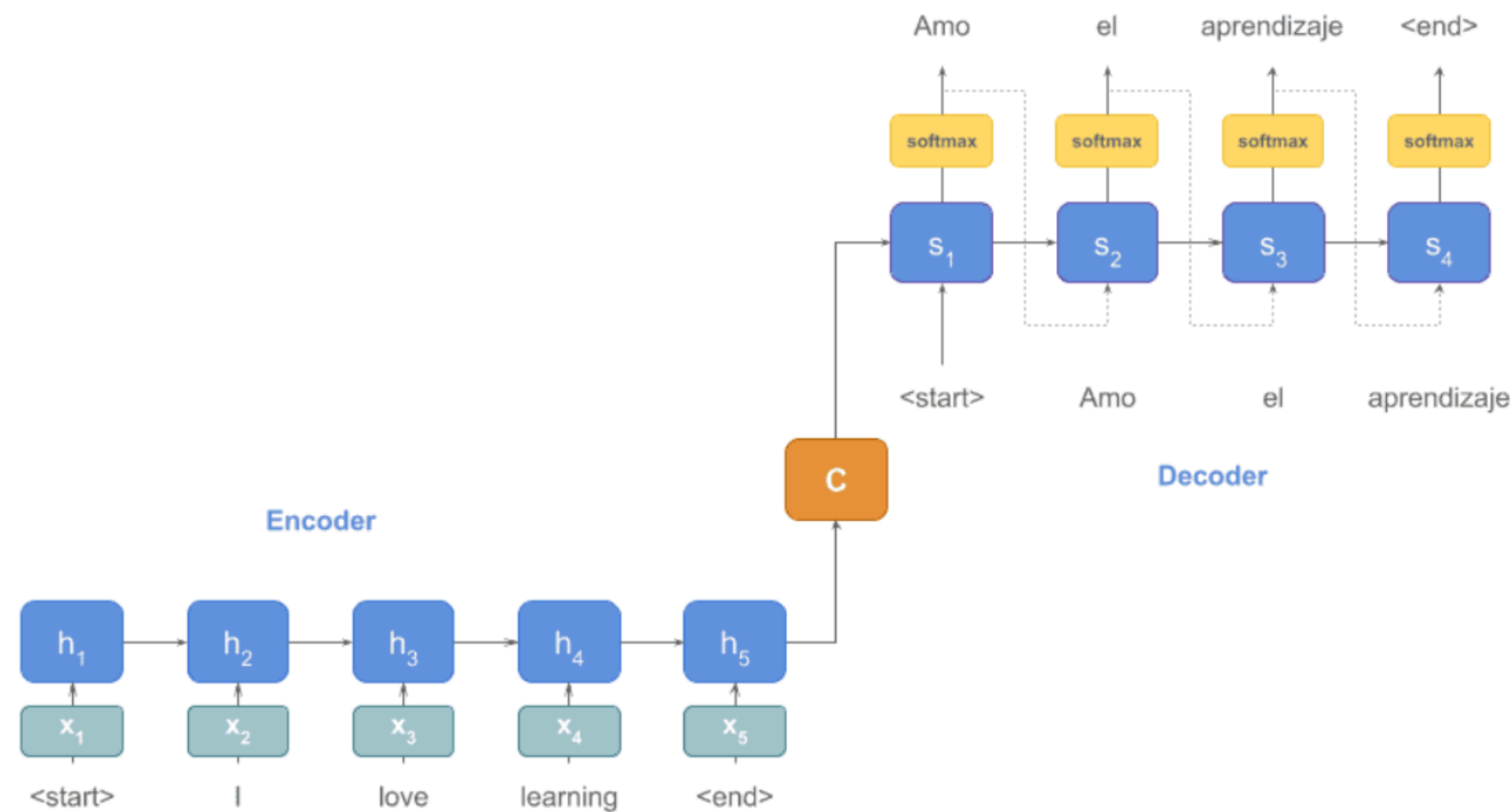


Block Input

Input Vector (encoding the word)
Hidden state vector (containing the sequence state before the current block)

Block Output

Output vector (which is not always produced by each block)



Applications

Machine Translation:

E.g., Google Translate.

Text Summarization:

Generating concise representations of documents.

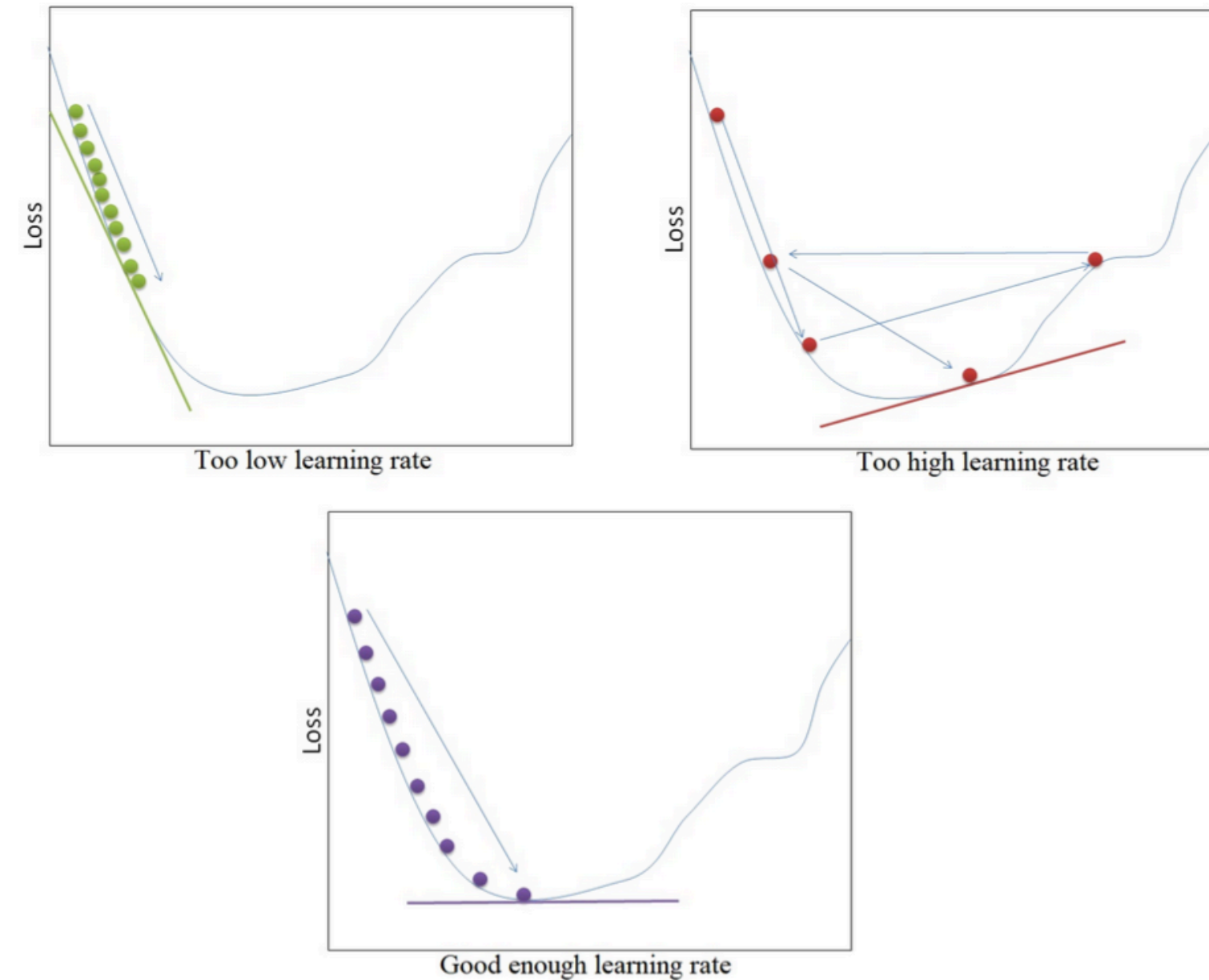
Speech-to-Text Conversion:

Used in virtual assistants like Siri.

Tuning Neural Network Hyperparameters and Layers

Tuning Hyperparameters: This refers to adjusting settings like learning rate, batch size, and optimizer to improve model performance during training. These parameters are set before training and impact how the model learns.

Adjusting Layers: This involves changing the architecture of the neural network by adding, removing, or modifying layers (e.g., dense, convolutional, or recurrent layers) to better capture patterns in the data.



The learning rate in deep learning controls the step size for the model to reach the minimum loss function. A higher learning rate speeds up learning but may overshoot the minimum, while a lower learning rate improves the chances of finding the minimum but requires more epochs, time, and memory.

Key Hyperparameters

Neurons per Layer

Range: 10–100 neurons.
Balance complexity:
Few neurons → Underfitting.
Too many neurons → Overfitting.

Learning Rate

High: Faster training, risks missing minima.
Low: Precise minima but slower learning.

Activation Functions

Define transformations between layers.
Choose from 9 types, each with unique behavior.

Batch Size

Controls training data size per iteration.
Small: Faster, higher variance.
Large: Slower, stable accuracy.

Optimizers

Adjust learning rates and weights.
Select from 7 optimizer options.

Epochs

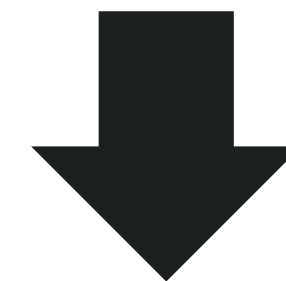
Number of full dataset passes.
Too few → Underfitting.
Too many → Overfitting.

Task 4

- Explored text summarization using LangChain.
- Used prompt engineering techniques with LangChain's prompt templates to achieve optimal results.
- Utilized PyPDF2 for direct summarization of text from PDFs.
- Summarized large documents using the MapReduce approach, both with and without custom prompts.
- Experimented with StuffDocumentChain for summarizing large chunks of text in one go.
- Applied RefineChain to iteratively refine and improve summaries, ensuring clarity and coherence.
- Achieved efficient summarization of complex documents through both chaining approaches.

```
[5] speech="""
People across the country, involved in government, political, and social activities, are dedicating their time to make the 'Viksit Bharat Sankalp Yatra' (Developed India Resolution Journey) successful. Therefore, as a Member of Parliament, it was my responsibility to also contribute my time to this program. So, today, I have come here just as a Member of Parliament and your 'sevak', ready to participate in this program, much like you. In our country, governments have come and gone, numerous schemes have been formulated, discussions have taken place, and big promises have been made. However, my experience and observations led me to believe that the most critical aspect that requires attention is ensuring that the government's plans reach the intended beneficiaries without any hassles. If there is a 'Pradhan Mantri Awas Yojana' (Prime Minister's housing scheme), then those who are living in jhuggis and slums should get their houses. And he should not need to make rounds of the government offices for this purpose. The government should reach him. Since you have assigned this responsibility to me, about four crore families have got their 'pucca' houses. However, I have encountered cases where someone is left out of the government benefits. Therefore, I have decided to tour the country again, to listen to people's experiences with government schemes, to understand whether they received the intended benefits, and to ensure that the programs are reaching everyone as planned without paying any bribes. We will get the real picture if we visit them again. Therefore, this 'Viksit Bharat Sankalp Yatra' is, in a way, my own examination. I want to hear from you and the people across the country whether what I envisioned and the work I have been doing aligns with reality and whether it has reached those for whom it was meant. It is crucial to check whether the work that was supposed to happen has indeed taken place. I recently met some individuals who utilized the Ayushman card to get treatment for serious illnesses. One person met with a severe accident, and after using the card, he could afford the necessary operation, and now he is recovering well. When I asked him, he said: "How could I afford this treatment? Now that there is the Ayushman card, I mustered courage and underwent an operation. Now I am perfectly fine." Such stories are blessings to me. The bureaucrats, who prepare good schemes, expedite the paperwork and even allocate funds, also feel satisfied that 50 or 100 people who were supposed to get the funds have got it. The funds meant for a thousand villages have been released. But their job satisfaction peaks when they hear that their work has directly impacted someone's life positively. When they see the tangible results of their efforts, their enthusiasm multiplies. They feel satisfied. Therefore, 'Viksit Bharat Sankalp Yatra' has had a positive impact on government officers. It has made them more enthusiastic about their work, especially when they witness the tangible benefits reaching the people. Officers now feel satisfied with their work, saying, "I made a good plan, I created a file, and the intended beneficiaries received the benefits." When they find that the money has reached a poor widow under the Jeevan Jyoti scheme and it was a great help to her during her crisis, they realise that they have done a good job. When a government officer listens to such stories, he feels very satisfied. There are very few who understand the power and impact of the 'Viksit Bharat Sankalp Yatra'. When I hear people connected to bureaucratic circles talking about it, expressing their satisfaction, it resonates with me. I've heard stories where someone suddenly received 2 lakh rupees after the death of her husband, and a sister mentioned how the arrival of gas in her home transformed her lives. The most significant aspect is when someone says that the line between rich and poor has vanished. While the slogan 'Garibi Hatao' (Remove Poverty) is one thing, but the real change happens when a person says, "As soon as the gas stove came to my house, the distinction between poverty and affluence disappeared."
"""
```

Speech



```
[9] llm(chat_messages).content
🔄 "In this speech, the speaker discusses the importance of ensuring that government plans and schemes reach the intended beneficiaries without any difficulties. They highlight their own efforts in providing housing to four crore families and express their commitment to touring the country to listen to people's experiences with government schemes. The speaker emphasizes the need to assess whether the work that was supposed to happen has actually taken place and shares stories of individuals who benefited from government initiatives such as the Ayushman card. They also discuss the positive impact of the 'Viksit Bharat Sankalp Yatra' on government officers, who feel satisfied when they see the tangible benefits of their efforts reaching people. The speaker concludes by highlighting the power and impact of the journey, as it helps bridge the gap between rich and poor and brings positive changes to people's lives."
```

Summarised Speech

Refer to next page for code Snippet

▼ Prompt Templates Text Summarization

```
[ ] from langchain.chains import LLMChain
    from langchain import PromptTemplate
```

```
[ ] generic_template=''
    Write a summary of the following speech:
    Speech : `{speech}`
    Translate the precise summary to {language}.

    ...

    prompt=PromptTemplate(
        input_variables=['speech','language'],
        template=generic_template
    )
```

```
[ ] prompt.format(speech=speech,language='Hindi')
```

 Show hidden output

```
[ ] complete_prompt=prompt.format(speech=speech,language='Hindi')
```

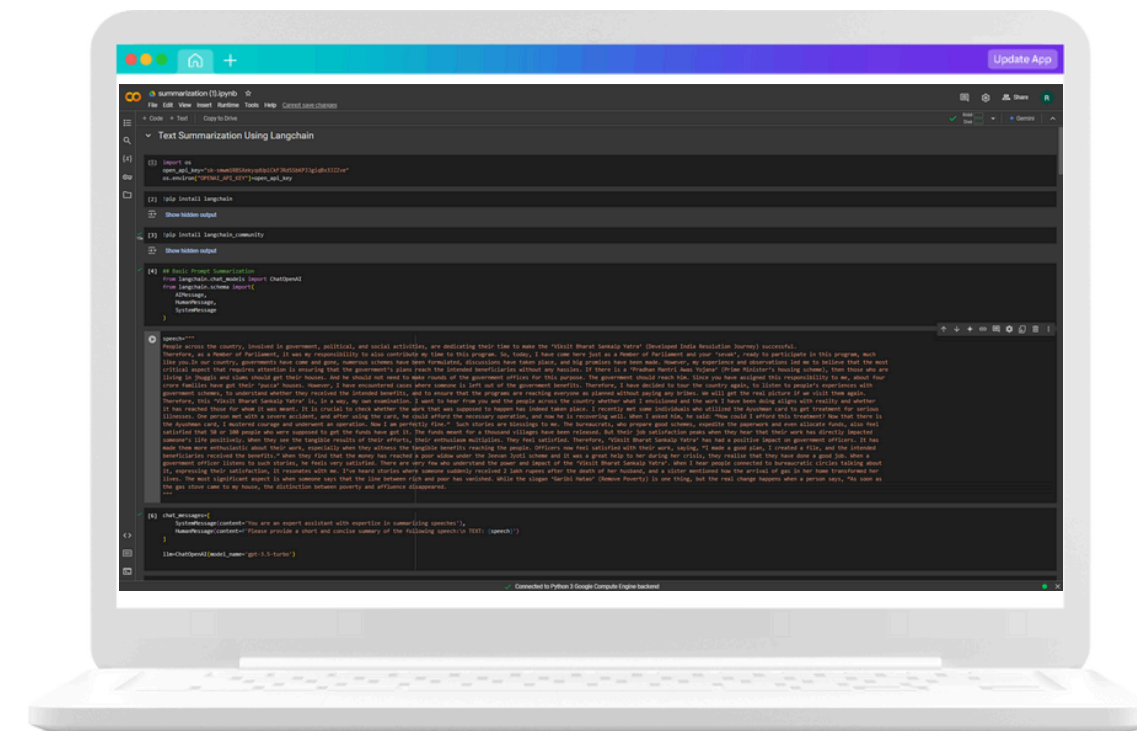
```
[ ] llm.get_num_tokens(complete_prompt)
```

 885

```
[ ] llm_chain=LLMChain(llm=llm,prompt=prompt)
    summary=llm_chain.run({'speech':speech,'language':'hindi'})
```

```
[ ] summary
```

- **Defines Prompt Template:** Creates a reusable structure to summarize a speech and translate it into a specific language.
- **Formats Prompt:** Replaces {speech} and {language} placeholders in the template with actual inputs (speech text and target language).
- **Calculates Token Count:** Checks the number of tokens in the formatted prompt to ensure it fits within the model's token limit.
- **Builds LLM Chain:** Combines the language model (LLM) and the formatted prompt for execution.
- **Executes Chain:** Runs the chain to generate a summary of the speech translated into the specified language (e.g., Hindi).
- **Stores Output:** The resulting translated summary is saved in the summary variable.




```
▼ StuffDocumentChain Text Summarization

[ ] from PyPDF2 import PdfReader

[ ] # provide the path of pdf file/files.
pdfreader = PdfReader('apjspeech.pdf')

[ ] from typing_extensions import Concatenate
# read text from pdf
text = ''
for i, page in enumerate(pdfreader.pages):
    content = page.extract_text()
    if content:
        text += content

[ ] text

⚡ Show hidden output

[ ] from langchain.docstore.document import Document

[ ] docs = [Document(page_content=text)]
docs

⚡ Show hidden output

[ ] llm = ChatOpenAI(temperature=0, model_name='gpt-3.5-turbo')
```

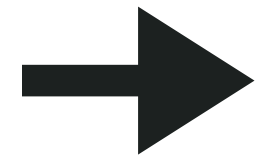
```
[ ] template = '''Write a concise and short summary of the following speech.
Speech: `{text}`
'''

prompt = PromptTemplate(
    input_variables=['text'],
    template=template
)

▶ chain = load_summarize_chain(
    llm,
    chain_type='stuff',
    prompt=prompt,
    verbose=False
)

output_summary = chain.run(docs)

▶ output_summary
```



- **Reads PDF:** Extracts text from all pages of the given PDF (apjspeech.pdf) using PyPDF2.
- **Prepares Document:** Wraps the extracted text into a LangChain Document object.
- **Defines Prompt:** Creates a template for summarizing the speech with LangChain's PromptTemplate.
- **Loads Summarization Chain:** Uses the "stuff" chain type with gpt-3.5-turbo to summarize the text.
- **Generates Summary:** Processes the document and produces a concise summary (output_summary).

```
▼ Map Reduce With Custom Prompts

[ ] chunks_prompt="""
Please summarize the below speech:
Speech: `{text}`
Summary:
"""

map_prompt_template=PromptTemplate(input_variables=['text'],
                                    template=chunks_prompt)

▶ final_combine_prompt='''
Provide a final summary of the entire speech with these important points.
Add a Generic Motivational Title,
Start the precise summary with an introduction and provide the
summary in number points for the speech.
Speech: `{text}`
'''

final_combine_prompt_template=PromptTemplate(input_variables=['text'],
                                             template=final_combine_prompt)

[ ] summary_chain = load_summarize_chain(
    llm=llm,
    chain_type='map_reduce',
    map_prompt=map_prompt_template,
    combine_prompt=final_combine_prompt_template,
    verbose=False
)

output = summary_chain.run(chunks)

[ ] output

⚡ 'Title: "Empowering India: Reflections and Aspirations"\n\nSummary:\n\n1. Dr. A. P. J. Abdul Kalam reflects on his time in office.\n2. He emphasizes the aspirations of the youth and the need to accelerate development in the digital age.\n3. The importance of innovation and competitiveness is discussed.\n4. Kalam shares his experiences with farmers and emphasizes the importance of agriculture.\n5. The role of entrepreneurship and partnership and courage is emphasized.\n6. The power of sea waves, his proposal for a Pan African e-Network, and the importance of infrastructure development.\n7. The need for a strong foundation in science and technology.\n8. Empowering the youth for a developed India by 2020 is highlighted.\n9. Preserving India's rich culture and heritage.\n10. Quality education, responsive governance, and eradication of poverty and crime.\n\nDr. Kalam's vision for India is a blend of tradition and modernity, aiming to create a strong, self-reliant, and inclusive nation.'''
```


**Thank you
very much!**